Medium Data Structures - C++ Solutions

■■■ ■■■■■■■■■: 10 ■■■■■■ ■■■■ ■■■■ + 3 Test Cases ■■■■ ■■■■■■ + ■■ ■■■■ C++.

Each section contains exactly 10 functions. Every function includes C++ code and 3 test cases.

1) ArrayList (Unsorted) - 10 Functions

1. removeAll

```
vector<int> removeAll(vector<int> a, int x) {
    int w = 0;
    for (int v : a) if (v != x) a[w++] = v;
    a.resize(w);
    return a;
}
```

Case	Input	Expected
1	a=[3,1,3,2], x=3	[1,2]
2	a=[5,5,5], x=5	[]
3	a=[1,2,3], x=9	[1,2,3]

2. deduplicateKeepFirst

```
vector<int> deduplicateKeepFirst(const vector<int>& a) {
    unordered_set<int> seen;
    vector<int> out;
    for (int v : a) if (seen.insert(v).second) out.push_back(v);
    return out;
}
```

Case	Input	Expected
1	a=[1,2,1,3,2]	[1,2,3]
2	a=[]	[]
3	a=[7,7,7]	[7]

3. rotateLeft

```
vector<int> rotateLeft(vector<int> a, int k) {
    if (a.empty()) return a;
    k %= (int)a.size();
    reverse(a.begin(), a.begin() + k);
    reverse(a.begin() + k, a.end());
    reverse(a.begin(), a.end());
    return a;
}
```

Case	Input	Expected
1	a=[1,2,3,4,5], k=2	[3,4,5,1,2]
2	a=[1], k=3	[1]
3	a=[1,2,3], k=0	[1,2,3]

4. moveZerosToEnd

```
vector<int> moveZerosToEnd(vector<int> a) {
    int w = 0;
    for (int v : a) if (v != 0) a[w++] = v;
    while (w < (int)a.size()) a[w++] = 0;
    return a;
}
```

Case	Input	Expected
1	a=[1,0,2,0,3]	[1,2,3,0,0]
2	a=[0,0,0]	[0,0,0]
3	a=[4,5]	[4,5]

5. maxSubarraySum

```
int maxSubarraySum(const vector<int>& a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        cur = max(a[i], cur + a[i]);
```

```
            best = max(best, cur);
        }
        return best;
    }
```

Case	Input	Expected
1	a=[-2,1,-3,4,-1,2,1,-5,4]	6
2	a=[1,2,3]	6
3	a=[-3,-1,-2]	-1

6. secondLargestDistinct

```
int secondLargestDistinct(const vector<int>& a) {
    long long m1 = LLONG_MIN, m2 = LLONG_MIN;
    for (int v : a) {
        if (v > m1) { m2 = m1; m1 = v; }
        else if (v < m1 && v > m2) m2 = v;
    }
    return (m2 == LLONG_MIN ? INT_MIN : (int)m2);
}
```

Case	Input	Expected
1	a=[5,1,9,7]	7
2	a=[4,4,4]	INT_MIN
3	a=[2,1]	1

7. productExceptSelf

```
vector<long long> productExceptSelf(const vector<int>& a) {
    int n = a.size();
    vector<long long> out(n,1);
    for (int i=1;i<n;++i) out[i]=out[i-1]*a[i-1];
    long long suf=1;
    for (int i=n-1;i>=0;--i){ out[i]*=suf; suf*=a[i]; }
    return out;
}
```

Case	Input	Expected
1	a=[1,2,3,4]	[24,12,8,6]
2	a=[0,1,2]	[2,0,0]
3	a=[-1,1,0,-3,3]	[0,0,9,0,0]

8. longestConsecutiveLength

```
int longestConsecutiveLength(const vector<int>& a) {
    unordered_set<int> s(a.begin(), a.end());
    int ans = 0;
    for (int v : s) if (!s.count(v-1)) {
        int x=v, len=1;
        while (s.count(x+1)) ++x, ++len;
        ans=max(ans,len);
    }
    return ans;
}
```

Case	Input	Expected
1	a=[100,4,200,1,3,2]	4
2	a=[1,2,0,1]	3
3	a=[]	0

9. kthSmallest

```
int kthSmallest(vector<int> a, int k) {
    nth_element(a.begin(), a.begin()+k-1, a.end());
    return a[k-1];
}
```

Case	Input	Expected

1	a=[7,10,4,3,20,15], k=3	7
2	a=[1], k=1	1
3	a=[5,2,9,1], k=4	9

10. partitionByPivot

```
vector<int> partitionByPivot(const vector<int>& a, int p) {
    vector<int> low, eq, high;
    for (int v : a) {
        if (v < p) low.push_back(v);
        else if (v == p) eq.push_back(v);
        else high.push_back(v);
    }
    low.insert(low.end(), eq.begin(), eq.end());
    low.insert(low.end(), high.begin(), high.end());
    return low;
}
```

Case	Input	Expected
1	a=[9,12,3,5,14,10,10], p=10	[9,3,5,10,10,12,14]
2	a=[1,2,3], p=0	[1,2,3]
3	a=[2,2,2], p=2	[2,2,2]

2) ArrayList (Sorted) - 10 Functions

1. lowerBound

```
int lowerBound(const vector<int>& a, int x) {
    int l=0,r=a.size();
    while(l<r){int m=(l+r)/2; if(a[m]<x) l=m+1; else r=m;}
    return l;
}
```

Case	Input	Expected
1	a=[1,2,2,4], x=2	1
2	a=[1,3,5], x=4	2
3	a=[], x=7	0

2. upperBound

```
int upperBound(const vector<int>& a, int x) {
    int l=0,r=a.size();
    while(l<r){int m=(l+r)/2; if(a[m]<=x) l=m+1; else r=m;}
    return l;
}
```

Case	Input	Expected
1	a=[1,2,2,4], x=2	3
2	a=[1,3,5], x=5	3
3	a=[], x=1	0

3. countEqual

```
int countEqual(const vector<int>& a, int x) {
    return upperBound(a,x) - lowerBound(a,x);
}
```

Case	Input	Expected
1	a=[1,2,2,2,4], x=2	3
2	a=[1,3,5], x=2	0
3	a=[7,7], x=7	2

4. insertSorted

```
vector<int> insertSorted(vector<int> a, int x) {
    int i = lowerBound(a, x);
    a.insert(a.begin()+i, x);
    return a;
}
```

Case	Input	Expected
1	a=[1,2,4], x=3	[1,2,3,4]
2	a=[], x=9	[9]
3	a=[2,2,2], x=2	[2,2,2,2]

5. deleteAllSorted

```
vector<int> deleteAllSorted(vector<int> a, int x) {
    int l=lowerBound(a,x), r=upperBound(a,x);
    a.erase(a.begin()+l, a.begin()+r);
    return a;
}
```

Case	Input	Expected
1	a=[1,2,2,3], x=2	[1,3]
2	a=[1,3,5], x=2	[1,3,5]

3	a=[4,4,4], x=4	[]

6. countInRange

```
int countInRange(const vector<int>& a, int L, int R) {
    if (L>R) return 0;
    return upperBound(a,R) - lowerBound(a,L);
}
```

Case	Input	Expected
1	a=[1,2,2,4,5], [2,4]	3
2	a=[1,1,1], [2,3]	0
3	a=[], [0,9]	0

7. firstGreater

```
int firstGreater(const vector<int>& a, int x) {
    int i = upperBound(a, x);
    return i == (int)a.size() ? -1 : a[i];
}
```

Case	Input	Expected
1	a=[1,2,4,4], x=3	4
2	a=[1,2,3], x=3	-1
3	a=[5], x=1	5

8. closestValueSorted

```
int closestValueSorted(const vector<int>& a, int x) {
    int i = lowerBound(a, x);
    if (i==0) return a[0];
    if (i==(int)a.size()) return a.back();
    return abs(a[i]-x) < abs(a[i-1]-x) ? a[i] : a[i-1];
}
```

Case	Input	Expected
1	a=[1,4,6,8], x=5	4 or 6 (tie choose 4 here)
2	a=[2,10], x=1	2
3	a=[2,10], x=20	10

9. mergeSorted

```
vector<int> mergeSorted(const vector<int>& a, const vector<int>& b) {
    vector<int> c; c.reserve(a.size()+b.size());
    int i=0,j=0;
    while(i<(int)a.size() && j<(int)b.size())
        c.push_back(a[i]<=b[j]?a[i++]:b[j++]);
    while(i<(int)a.size()) c.push_back(a[i++]);
    while(j<(int)b.size()) c.push_back(b[j++]);
    return c;
}
```

Case	Input	Expected
1	a=[1,3,5], b=[2,4,6]	[1,2,3,4,5,6]
2	a=[], b=[1,2]	[1,2]
3	a=[1,1], b=[1]	[1,1,1]

10. intersectionSorted

```
vector<int> intersectionSorted(const vector<int>& a, const vector<int>& b) {
    vector<int> out; int i=0,j=0;
    while(i<(int)a.size() && j<(int)b.size()){
        if(a[i]==b[j]){ out.push_back(a[i]); ++i; ++j; }
        else if(a[i]<b[j]) ++i; else ++j;
    }
    return out;
}
```

Case	Input	Expected
1	a=[1,2,2,4], b=[2,2,3]	[2,2]
2	a=[1,3], b=[2,4]	[]
3	a=[], b=[1]	[]

3) LinkedList (Singly) - 10 Functions

1. findMiddle

```
ListNode* findMiddle(ListNode* head){
    ListNode *slow=head,*fast=head;
    while(fast && fast->next){ slow=slow->next; fast=fast->next->next; }
    return slow;
}
```

Case	Input	Expected
1	1->2->3->4->5	3
2	1->2->3->4	3
3	1	1

2. removeNthFromEnd

```
ListNode* removeNthFromEnd(ListNode* head,int n){
    ListNode dummy(0,head); auto fast=&dummy; auto slow=&dummy;
    for(int i=0;i<n;i++) fast=fast->next;
    while(fast->next){ fast=fast->next; slow=slow->next; }
    slow->next=slow->next->next; return dummy.next;
}
```

Case	Input	Expected
1	1->2->3->4->5, n=2	1->2->3->5
2	1, n=1	empty
3	1->2, n=2	2

3. reverseList

```
ListNode* reverseList(ListNode* head){
    ListNode *prev=nullptr,*cur=head;
    while(cur){ auto nxt=cur->next; cur->next=prev; prev=cur; cur=nxt; }
    return prev;
}
```

Case	Input	Expected
1	1->2->3	3->2->1
2	1	1
3	empty	empty

4. isPalindromeList

```
bool isPalindromeList(ListNode* head){
    if(!head||!head->next) return true;
    ListNode *slow=head,*fast=head;
    while(fast->next&&fast->next->next){ slow=slow->next; fast=fast->next->next; }
    ListNode* second=reverseList(slow->next);
    for(ListNode *p1=head,*p2=second;p2;p1=p1->next,p2=p2->next)
        if(p1->val!=p2->val) return false;
    return true;
}
```

Case	Input	Expected
1	1->2->2->1	true
2	1->2	false
3	1	true

5. detectCycle

```
bool detectCycle(ListNode* head){
    ListNode *slow=head,*fast=head;
    while(fast&&fast->next){ slow=slow->next; fast=fast->next->next; if(slow==fast) return true; }
    return false;
}
```

Case	Input	Expected
1	1->2->3->2(cycle)	true
2	1->2->3	false
3	empty	false

6. mergeTwoSortedLists

```
ListNode* mergeTwoSortedLists(ListNode* a,ListNode* b){
    ListNode dummy; auto t=&dummy;
    while(a&&b){ if(a->val<=b->val){t->next=a;a=a->next;} else {t->next=b;b=b->next;} t=t->next; }
    t->next=a?a:b; return dummy.next;
}
```

Case	Input	Expected
1	1->3->5 + 2->4->6	1->2->3->4->5->6
2	empty + 1->2	1->2
3	1->1 + 1	1->1->1

7. sortListMergeSort

```
ListNode* sortListMergeSort(ListNode* head){
    if(!head||!head->next) return head;
    ListNode *slow=head,*fast=head->next;
    while(fast&&fast->next){ slow=slow->next; fast=fast->next->next; }
    ListNode* mid=slow->next; slow->next=nullptr;
    return mergeTwoSortedLists(sortListMergeSort(head), sortListMergeSort(mid));
}
```

Case	Input	Expected
1	4->2->1->3	1->2->3->4
2	-1->5->3->4->0	-1->0->3->4->5
3	1	1

8. partitionList

```
ListNode* partitionList(ListNode* head,int x){
    ListNode lDummy,hDummy; auto l=&lDummy; auto h=&hDummy;
    while(head){ if(head->val<x){l->next=head;l=l->next;} else {h->next=head;h=h->next;} head=head->next; }
    h->next=nullptr; l->next=hDummy.next; return lDummy.next;
}
```

Case	Input	Expected
1	1->4->3->2->5->2, x=3	1->2->2->4->3->5
2	2->1, x=2	1->2
3	empty, x=0	empty

9. oddEvenList

```
ListNode* oddEvenList(ListNode* head){
    if(!head) return head;
    ListNode *odd=head,*even=head->next,*evenHead=even;
    while(even&&even->next){ odd->next=even->next; odd=odd->next; even->next=odd->next; even=even->next; }
    odd->next=evenHead; return head;
}
```

Case	Input	Expected
1	1->2->3->4->5	1->3->5->2->4
2	2->1->3->5->6->4->7	2->3->6->7->1->5->4
3	1	1

10. removeDuplicatesSortedList

```
ListNode* removeDuplicatesSortedList(ListNode* head){
    auto cur=head;
```

```
        while(cur&&cur->next){
            if(cur->val==cur->next->val) cur->next=cur->next->next;
            else cur=cur->next;
        }
        return head;
    }
```

Case	Input	Expected
1	1->1->2->3->3	1->2->3
2	1->1->1	1
3	1->2->3	1->2->3

4) Stack - 10 Functions

1. isBalanced

```
bool isBalanced(const string& s){
    stack<char> st;
    unordered_map<char,char> mp={{')','('},{']','['},{'}','{'}};
    for(char c:s){
        if(c=='('||c=='['||c=='{') st.push(c);
        else if(mp.count(c)){ if(st.empty()||st.top()!=mp[c]) return false; st.pop(); }
    }
    return st.empty();
}
```

Case	Input	Expected
1	"{[()]}"	true
2	"([)]"	false
3	""	true

2. evaluatePostfix

```
int evaluatePostfix(const string& s){
    stack<int> st;
    for(char c:s){
        if(isdigit(c)) st.push(c-'0');
        else{ int b=st.top(); st.pop(); int a=st.top(); st.pop();
            if(c=='+') st.push(a+b); else if(c=='-') st.push(a-b);
            else if(c=='*') st.push(a*b); else st.push(a/b);
        }
    }
    return st.top();
}
```

Case	Input	Expected
1	"23*54*+9-"	17
2	"82/"	4
3	"34+5*"	35

3. nextGreaterElement

```
vector<int> nextGreaterElement(const vector<int>& a){
    int n=a.size(); vector<int> ans(n,-1); stack<int> st;
    for(int i=0;i<n;i++){
        while(!st.empty()&&a[i]>a[st.top()]){ ans[st.top()]=a[i]; st.pop(); }
        st.push(i);
    }
    return ans;
}
```

Case	Input	Expected
1	a=[2,1,2,4,3]	[4,2,4,-1,-1]
2	a=[4,3,2,1]	[-1,-1,-1,-1]
3	a=[1,3,2,4]	[3,4,4,-1]

4. stockSpan

```
vector<int> stockSpan(const vector<int>& p){
    vector<int> span(p.size()); stack<int> st;
    for(int i=0;i<(int)p.size();++i){
        while(!st.empty()&&p[st.top()]<=p[i]) st.pop();
        span[i]=st.empty()?i+1:i-st.top();
        st.push(i);
    }
    return span;
}
```

Case	Input	Expected
1	p=[100,80,60,70,60,75,85]	[1,1,1,2,1,4,6]

2	p=[10,20,30]	[1,2,3]
3	p=[30,20,10]	[1,1,1]

5. minStackPushPopTopMin

```
class MinStack {
    stack<int> st,mn;
public:
    void push(int x){ st.push(x); mn.push(mn.empty()?x:min(x,mn.top())); }
    void pop(){ st.pop(); mn.pop(); }
    int top(){ return st.top(); }
    int getMin(){ return mn.top(); }
};
```

Case	Input	Expected
1	ops: push(3),push(5),push(2),getMin	2
2	after pop(), getMin	3
3	push(-1), getMin	-1

6. removeKdigits

```
string removeKdigits(string num, int k){
    string st;
    for(char c:num){
        while(k && !st.empty() && st.back()>c){ st.pop_back(); --k; }
        st.push_back(c);
    }
    while(k-- && !st.empty()) st.pop_back();
    int i=0; while(i<(int)st.size()&&st[i]=='0') ++i;
    string ans=st.substr(i);
    return ans.empty()?"0":ans;
}
```

Case	Input	Expected
1	num="1432219", k=3	1219
2	num="10200", k=1	200
3	num="10", k=2	0

7. simplifyPath

```
string simplifyPath(const string& path){
    vector<string> st; string cur;
    for(int i=0;i<=path.size();++i){
        if(i==path.size()||path[i]=='/'){
            if(cur==".."&&!st.empty()) st.pop_back();
            else if(!cur.empty()&&cur!="."&&cur!="..") st.push_back(cur);
            cur.clear();
        } else cur.push_back(path[i]);
    }
    string out="/";
    for(int i=0;i<(int)st.size();++i){ if(i) out+='/'; out+=st[i]; }
    return out;
}
```

Case	Input	Expected
1	"/a/./b/../../c/"	/c
2	"/../"	/
3	"/home//foo/"	/home/foo

8. largestRectangleHistogram

```
int largestRectangleHistogram(const vector<int>& h){
    stack<int> st; int ans=0, n=h.size();
    for(int i=0;i<=n;++i){
        int cur=(i==n?0:h[i]);
        while(!st.empty()&&h[st.top()]>cur){
            int ht=h[st.top()]; st.pop();
            int l=st.empty()?0:st.top()+1;
            ans=max(ans, ht*(i-l));
        }
```

```
            st.push(i);
        }
        return ans;
    }
```

Case	Input	Expected
1	h=[2,1,5,6,2,3]	10
2	h=[2,4]	4
3	h=[1,1,1,1]	4

9. decodeString

```
string decodeString(const string& s){
    stack<int> num; stack<string> str; string cur=""; int k=0;
    for(char c:s){
        if(isdigit(c)) k=k*10+(c-'0');
        else if(c=='['){ num.push(k); str.push(cur); k=0; cur=""; }
        else if(c==']'){ int t=num.top(); num.pop(); string prev=str.top(); str.pop(); while(t--) prev+=cur; c
        else cur.push_back(c);
    }
    return cur;
}
```

Case	Input	Expected
1	"3[a2[c]]"	accaccacc
2	"2[abc]3[cd]ef"	abcabccdcdcdef
3	"10[a]"	aaaaaaaaaa

10. dailyTemperatures

```
vector<int> dailyTemperatures(const vector<int>& t){
    int n=t.size(); vector<int> ans(n,0); stack<int> st;
    for(int i=0;i<n;++i){
        while(!st.empty()&&t[i]>t[st.top()]){ int j=st.top(); st.pop(); ans[j]=i-j; }
        st.push(i);
    }
    return ans;
}
```

Case	Input	Expected
1	t=[73,74,75,71,69,72,76,73]	[1,1,4,2,1,1,0,0]
2	t=[30,40,50,60]	[1,1,1,0]
3	t=[30,60,90]	[1,1,0]

5) Queue / Deque - 10 Functions

1. reverseQueue

```
queue<int> reverseQueue(queue<int> q){
    stack<int> st;
    while(!q.empty()){ st.push(q.front()); q.pop(); }
    while(!st.empty()){ q.push(st.top()); st.pop(); }
    return q;
}
```

Case	Input	Expected
1	q=[1,2,3,4]	[4,3,2,1]
2	q=[5]	[5]
3	q=[]	[]

2. reverseFirstK

```
queue<int> reverseFirstK(queue<int> q, int k){
    stack<int> st; int n=q.size();
    for(int i=0;i<k;i++){ st.push(q.front()); q.pop(); }
    while(!st.empty()){ q.push(st.top()); st.pop(); }
    for(int i=0;i<n-k;i++){ q.push(q.front()); q.pop(); }
    return q;
}
```

Case	Input	Expected
1	q=[1,2,3,4,5], k=3	[3,2,1,4,5]
2	q=[1,2], k=2	[2,1]
3	q=[7,8,9], k=1	[7,8,9]

3. queueRotate

```
queue<int> queueRotate(queue<int> q, int k){
    int n=q.size(); if(!n) return q; k%=n;
    while(k--){ q.push(q.front()); q.pop(); }
    return q;
}
```

Case	Input	Expected
1	q=[1,2,3,4], k=1	[2,3,4,1]
2	q=[1,2,3,4], k=4	[1,2,3,4]
3	q=[9], k=10	[9]

4. interleaveHalves

```
queue<int> interleaveHalves(queue<int> q){
    int n=q.size(); queue<int> a,b;
    for(int i=0;i<n/2;i++){ a.push(q.front()); q.pop(); }
    while(!q.empty()){ b.push(q.front()); q.pop(); }
    while(!a.empty()||!b.empty()){
        if(!a.empty()){ q.push(a.front()); a.pop(); }
        if(!b.empty()){ q.push(b.front()); b.pop(); }
    }
    return q;
}
```

Case	Input	Expected
1	q=[1,2,3,4]	[1,3,2,4]
2	q=[1,2,3,4,5,6]	[1,4,2,5,3,6]
3	q=[1,2]	[1,2]

5. firstNegativeInWindow

```
vector<int> firstNegativeInWindow(const vector<int>& a, int k){
    deque<int> dq; vector<int> ans;
```

```
    for(int i=0;i<(int)a.size();++i){
        if(a[i]<0) dq.push_back(i);
        if(i>=k-1){
            while(!dq.empty()&&dq.front()<=i-k) dq.pop_front();
            ans.push_back(dq.empty()?0:a[dq.front()]);
        }
    }
    return ans;
}
```

Case	Input	Expected
1	a=[12,-1,-7,8,-15,30,16,28], k=3	[-1,-1,-7,-15,-15,0]
2	a=[1,2,3], k=2	[0,0]
3	a=[-5,-2,-3], k=2	[-5,-2]

6. slidingWindowMaximum

```
vector<int> slidingWindowMaximum(const vector<int>& a, int k){
    deque<int> dq; vector<int> ans;
    for(int i=0;i<(int)a.size();++i){
        while(!dq.empty()&&dq.front()<=i-k) dq.pop_front();
        while(!dq.empty()&&a[dq.back()]<=a[i]) dq.pop_back();
        dq.push_back(i);
        if(i>=k-1) ans.push_back(a[dq.front()]);
    }
    return ans;
}
```

Case	Input	Expected
1	a=[1,3,-1,-3,5,3,6,7], k=3	[3,3,5,5,6,7]
2	a=[9,8,7], k=2	[9,8]
3	a=[4], k=1	[4]

7. generateBinaryNumbers

```
vector<string> generateBinaryNumbers(int n){
    vector<string> out; queue<string> q; q.push("1");
    while(n--){ string s=q.front(); q.pop(); out.push_back(s); q.push(s+"0"); q.push(s+"1"); }
    return out;
}
```

Case	Input	Expected
1	n=5	[1,10,11,100,101]
2	n=1	[1]
3	n=3	[1,10,11]

8. timeToRotAllOranges

```
int timeToRotAllOranges(vector<vector<int>> g){
    int n=g.size(),m=g[0].size(),fresh=0,mins=0; queue<pair<int,int>> q;
    for(int i=0;i<n;i++) for(int j=0;j<m;j++){
        if(g[i][j]==2) q.push({i,j}); else if(g[i][j]==1) fresh++;
    }
    int d[5]={-1,0,1,0,-1};
    while(!q.empty()&&fresh){
        int sz=q.size(); mins++;
        while(sz--){ auto [x,y]=q.front(); q.pop();
            for(int k=0;k<4;k++){ int nx=x+d[k], ny=y+d[k+1];
                if(nx>=0&&ny>=0&&nx<n&&ny<m&&g[nx][ny]==1){ g[nx][ny]=2; fresh--; q.push({nx,ny}); }
            }
        }
    }
    return fresh?-1:mins;
}
```

Case	Input	Expected
1	grid=[[2,1,1],[1,1,0],[0,1,1]]	4
2	grid=[[2,1,1],[0,1,1],[1,0,1]]	-1
3	grid=[[0,2]]	0

9. canCompleteCircuit

```
int canCompleteCircuit(const vector<int>& gas,const vector<int>& cost){
    int total=0,cur=0,start=0;
    for(int i=0;i<(int)gas.size();++i){
        int diff=gas[i]-cost[i]; total+=diff; cur+=diff;
        if(cur<0){ cur=0; start=i+1; }
    }
    return total<0?-1:start;
}
```

Case	Input	Expected
1	gas=[1,2,3,4,5], cost=[3,4,5,1,2]	3
2	gas=[2,3,4], cost=[3,4,3]	-1
3	gas=[5], cost=[4]	0

10. leastIntervalTasks

```
int leastIntervalTasks(vector<char> tasks, int n){
    vector<int> f(26,0); for(char c:tasks) f[c-'A']++;
    int mx=*max_element(f.begin(),f.end());
    int cnt=count(f.begin(),f.end(),mx);
    return max((int)tasks.size(), (mx-1)*(n+1)+cnt);
}
```

Case	Input	Expected
1	tasks=[A,A,A,B,B,B], n=2	8
2	tasks=[A,A,A,B,B,B], n=0	6
3	tasks=[A,A,A,A,B,C,D,E], n=2	10